

孔力帆答辩闭环：类型系统与错误诊断支持

1. 先记住自己的定位

你这部分的核心不是“把整个语义分析讲完”，而是守住两件事：

1. 类型查询为什么要独立成一层，以及它具体查询什么。
2. 错误信息怎么从 AST 节点位置变成带源码行和 `^~~~` 的 `caret diagnostics`。

最短自我介绍可以直接说：

我主要参与的是类型系统与错误诊断支持这一层。类型查询模块负责统一推断表达式、数组和 `record` 字段的类型，避免语义分析和代码生成各写一套规则；错误诊断模块负责把行列号和源码文本组织成可读的 `caret diagnostics` 输出。

2. 现场先打开哪些文件

建议只记这 3 个文件，够用了：

1. [src/type_resolver.h](#)
2. [src/type_resolver.cpp](#)
3. [src/error.cpp](#)

如果老师顺着问“这和语义分析怎么接上”，再补开：

4. [src/semantic_analyzer.cpp](#)

打开顺序建议：

1. 先看 `ResolvedType` 和 `resolve_expr_type()`，讲“类型查询是什么”。
2. 再看 `resolve_record_info()` 和 `find_record_field()`，讲“record 字段类型怎么找”。
3. 最后看 `ErrorHandler::print_errors()`，讲“错误怎么打印成 `caret diagnostics`”。

3. 老师最可能问什么

问题 A：你们的类型系统支持哪些类型？

先打开：

- [src/type_resolver.h:10](#)
- [src/type_resolver.h:22](#)

回答模板：

这里支持的基础类型包括 `integer`、`real`、`boolean`、`char`，另外还支持数组、`record`、函数和过程相关的类型信息。

`SymbolTypeInfo` 用来描述一个符号在符号表里的类型属性，`ResolvedType` 用来描述一个表达式在当前上下文里被解析成什么类型。

如果老师追问“为什么分成两个结构”：

因为符号和表达式不是一个层次。符号有名字、返回类型、数组信息这些声明属性；表达式只需要知道最终解析出的类型结果以及必要的数组/record 附加信息。

问题 B：为什么还要单独做 `TypeResolver`？不是语义分析直接查就行吗？

先打开：

- [src/type_resolver.cpp:6](#)
- [src/type_resolver.cpp:50](#)

回答模板：

`TypeResolver` 的作用是把类型查询统一起来。

如果不拆出来，语义分析要自己写一套表达式类型推断，代码生成又可能再写一套，这样很容易规则不一致。

现在 `resolve_expr_type()` 专门回答“这个表达式在当前上下文里是什么类型”，而“这个类型是否合法”仍然由语义分析决定。

一句最关键的话：

类型查询和类型检查不是一回事：`TypeResolver` 做前者，`SemanticAnalyzer` 做后者。

问题 C：record 字段类型是怎么查出来的？

先打开：

- [src/type_resolver.cpp:7](#)
- [src/type_resolver.cpp:16](#)
- [src/type_resolver.cpp:93](#)

回答模板：

`record` 字段查询分两步。

第一步，`resolve_record_info()` 先递归确认左边表达式到底对应哪个 `record` 类型。

第二步，`find_record_field()` 再到字段表里按名字查具体字段。

所以像 `point.inner.y` 这种多级字段访问，本质上就是递归先找到 `point.inner` 的 `record` 信息，再查 `y`。

如果老师追问“字段不存在时在哪儿报错”：

字段查询函数只负责返回结果，真正发现“字段不存在”时，是语义分析阶段根据空结果去报 `Unknown record field`。

问题 D：表达式类型是怎么推断的？

先打开：

- [src/type_resolver.cpp:50](#)

回答模板：

`resolve_expr_type()` 会按表达式节点种类分别处理。

字面量直接返回固定类型；标识符通过符号表查；数组访问返回元素类型；`record` 字段访问返回字段类型；一元和二元运算则根据操作数类型和运算符规则推断结果类型；函数调用返回函数返回类型。

如果老师追问“`div/mod` 和 `/` 有什么区别”：

`div/mod` 要求整数并返回整数，而 Pascal 里的 `/` 表示实数除法，所以最后会提升到 `real`。

问题 E：错误诊断是怎么做到源码行 + 箭头的？

先打开：

- [src/error.h:29](#)
- [src/error.cpp:11](#)
- [src/error.cpp:67](#)

回答模板：

错误处理模块里会先缓存整份源文件的每一行文本，也就是 `source_lines`。

当语法或语义阶段报错时，不只记录错误消息，还会记录行号、列号和长度。

最后 `print_errors()` 输出时，会先打印出错源码行，再按列号补空格，最后打印 `^` 和若干个 `~`，形成 `caret diagnostics`。

这段一定要点这几个概念：

- `source_lines`
- `line`
- `column`
- `length`
- `print_errors()`

问题 F：你们的 panic-mode 恢复是怎么回事？

先打开：

- [src/error.h:57](#)
- [src/error.cpp:31](#)
- [src/error.cpp:40](#)

标准答法：

这里的 `panic_mode` 主要是错误诊断层面的辅助状态，用来抑制同一处语法错误引发的重复报错。

真正的 panic-mode 恢复点是在 parser 里的 `error` 产生式和同步点，比如分号、`END`、`UNTIL`。

所以这里负责的是“不要同一处错误刷屏”，而不是在 `error.cpp` 里直接吃 token。

这题容易说混，一定要记：

parser 做恢复，`ErrorHandler` 做抑制重复诊断和格式化输出。

4. 最该会指给老师看的 5 段代码

4.1 `ResolvedType`

位置:

- [src/type_resolver.h:22](#)

怎么讲:

这是表达式类型查询的结果结构。它除了基本 `type`，还会带数组和 `record` 的附加信息，便于后面继续做字段查询和代码生成。

4.2 `resolve_expr_type()`

位置:

- [src/type_resolver.cpp:50](#)

怎么讲:

这段是类型查询总入口。它不判断“对不对”，只判断“当前表达式被看作什么类型”。语义分析和代码生成都复用这一套规则。

4.3 `resolve_record_info()` + `find_record_field()`

位置:

- [src/type_resolver.cpp:7](#)
- [src/type_resolver.cpp:16](#)

怎么讲:

这两段配合起来处理 `record`。一个负责递归找到当前 `record` 类型，一个负责在字段表中按名字查字段。

4.4 `ErrorHandler::print_errors()`

位置:

- [src/error.cpp:67](#)

怎么讲:

这段是 `caret diagnostics` 的核心。它先输出错误消息，再输出源码原文，然后根据列号打印 `^~~~`。

4.5 `SemanticAnalyzer::report_semantic_error()`

位置:

- [src/semantic_analyzer.cpp:40](#)

怎么讲:

语义分析阶段不是直接打印字符串,而是把节点的行、列、诊断长度统一交给 `ErrorHandler`。所以类型错误、字段错误、参数错误都能用同一套 `caret_diagnostics` 输出出来。

5. 如果老师一个点一个点追问, 怎么答

5.1 `TypeResolver` 和 `SemanticAnalyzer` 的边界是什么?

回答:

`TypeResolver` 只做类型查询和类型推断, `SemanticAnalyzer` 再利用这些结果做合法性检查。也就是一个回答“是什么类型”, 另一个回答“这样用是否允许”。

5.2 为什么错误里还要存 `length`?

回答:

因为只知道列号还不够, 我们希望箭头尽量覆盖真正出错的名字或字段, 所以除了 `^` 之外还会按长度补 `~`, 这样定位更直观。

5.3 为什么要对同一行列去重?

位置:

- [src/error.cpp:102](#)

回答:

因为一处错误可能级联触发多个后续诊断, 如果不去重, 输出会刷屏。这里按行列去重, 是为了让错误信息更可读。

5.4 `caret_diagnostics` 是词法、语法还是语义阶段做的?

回答:

三个阶段都可能触发, 但格式化输出统一在 `ErrorHandler`。词法阶段提供行列, 语法和语义阶段给出错误类型和长度, 最后用同一套输出方式展示。

6. 老师如果让你“讲设计思路”，你就这么收

我们把类型查询和错误输出都做成了公共支撑层。类型查询统一由 `TypeResolver` 负责，避免语义分析和代码生成各写一套规则；错误输出统一由 `ErrorHandler` 负责，保证词法、语法、语义阶段都能落成一致的 `caret diagnostics` 格式。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我主要参与的是类型系统与错误诊断支持。`TypeResolver` 统一负责表达式、数组和 `record` 字段的类型查询，避免语义分析和代码生成重复实现；`ErrorHandler` 统一负责行列号、源码行和 `^~~~` 箭头定位的错误输出。相关代码主要在 `type_resolver.*` 和 `error.*`。

8. 最容易说错的 4 个点

1. 不要把类型查询说成类型检查。
正确说法：类型查询回答“是什么类型”，类型检查回答“是否合法”。
2. 不要把 panic-mode 恢复全说成 `ErrorHandler` 做的。
正确说法：parser 负责恢复，`ErrorHandler` 负责抑制重复诊断和统一输出。
3. 不要把 `record -> struct` 说成你这一层做的。
正确说法：你这一层只负责找到 `record` 字段和类型信息，`struct` 映射在 codegen。
4. 不要把 `caret diagnostics` 说成只靠语义分析。
正确说法：词法、语法、语义阶段都提供信息，最后由 `ErrorHandler` 统一格式化。

9. 孔力帆只需要记住的文件定位

最推荐只记下面这些：

- [src/type_resolver.h:22](#)
- [src/type_resolver.cpp:16](#)
- [src/type_resolver.cpp:50](#)
- [src/error.h:29](#)
- [src/error.cpp:67](#)
- [src/semantic_analyzer.cpp:40](#)

这 6 处足够覆盖他的大部分答辩问题。